

Checkpoint 3.4 — Consolidated Data Schemas

Amen Bank — Real-Time Anomaly Detection Platform

Internship Project Documentation

June 3, 2026

1 Purpose

This document is the **consolidated reference** for every data schema in the system. It supersedes the per-schema views in Checkpoints 3, 3.2, and 3.3, gathering everything in one place for implementation reference.

Four schemas are defined:

1. **Raw event schema** — the input to the pipeline (**raw-events** topic).
2. **Client profile schema** — hot (Redis) + cold (PostgreSQL).
3. **Employee profile schema** — hot (Redis) + cold (PostgreSQL).
4. **Enriched event schema** — the output of the Enrichment Service (**enriched-events** topic), consumed by the Scoring Service.

2 Data Flow Overview

Streaming path:

raw-events (raw event) → Enrichment Service
→ looks up client profile (Redis)
→ looks up employee profile (Redis)
→ computes derived features
→ **enriched-events** (enriched event)
→ Scoring Service (Autoencoder + LSTM + SHAP)
→ **scored-events** → Decision Service → **decisions**

Batch path:

PostgreSQL (**transactions**, **alerts**, **clients_master**, **accounts**, **account_balances**, **employees_master**)
→ Profile Manager (nightly)
→ Recomputes profiles (cold + hot stores)
→ Pushes refreshed hot profiles to Redis

3 Schema 1 — Raw Event

Pattern: Tagged-union envelope. Single **raw-events** topic covers all four operation types, partitioned by **client_id** to preserve cross-operation client ordering for the LSTM.

Principles:

- Raw events carry **only raw facts**. No statistics, no aggregations, no derived/lagging fields.
- **remise_cheque** produces **one event per check**, linked by a shared **remise_id**.

3.1 Common Envelope

```
raw_event {
  event_id      : string (UUID)
  client_id     : string
  account_id    : string
  employee_id   : string
  branch_id     : string
  timestamp     : datetime
  amount        : float
  currency      : string (typically TND)
  channel       : enum    [guichet] // scope: teller only
  operation_type : enum    [retrait, versement, virement, cheque]
  payload       : object  // per operation type, see below
}
```

3.2 Operation-Specific Payloads

```
// retrait
payload_retrait {
  id_verified      : bool
  is_third_party   : bool
  third_party_id   : string (nullable)
}

// versement
payload_versement {
  is_third_party_depositor : bool
  depositor_id            : string (nullable)
  source_of_funds         : string
  source_verified         : bool
}

// virement
payload_virement {
  beneficiary_account : string
  beneficiary_name    : string
  transfer_motif      : string
  is_internal_transfer : bool
}

// remise_cheque (one event PER CHECK)
payload_cheque {
  remise_id          : string (shared across same remise)
  check_number       : string
  emitter_account    : string
  emitter_name       : string
  emitter_bank       : string
  check_date         : date
  is_endorsed_to_depositor : bool
}
```

4 Schema 2 — Client Profile

4.1 Hot Profile (Redis) — 6 Categories, ~194 fields

```

client_profile_hot {

    // --- 1. Personal Info (7) ---
    client_id          : string   (PK)
    archetype          : enum     [student, salaried, small_biz,
                                   retiree, big_biz]
    maturity_status    : enum     [cold_start, mature]
    home_branch_id     : string
    client_type        : enum     [individual, business]
    nationality         : string
    account_opening_date : date

    // --- 2. Transaction Statistics (113) ---
    days_since_last_transaction : int

    // Per op_type x window
    // op_type in {retrait, versement, virement, cheque}
    // window in {24h, 7d, 30d, 90d}
    stats_<op>_<window> {
        count, sum_amount, mean_amount, std_amount,
        rejected_count, reversed_count, near_threshold_ratio
    }

    // --- 3. Behavioral Patterns (12) ---
    behavior_30d {
        branch_distribution, hour_distribution, dow_distribution,
        operation_distribution, employee_distribution
    }
    behavior_180d { ... same 5 distributions ... }

    known_beneficiaries_365d : {
        beneficiary_account -> {first_seen, count, total_amount, last_seen}
    }
    known_emitters_365d : {
        emitter_account -> {first_seen, count, total_amount, last_seen}
    }

    // --- 4. Financial State (12) ---
    current_balance, current_debt, num_accounts,
    account_types_distribution,
    avg_balance_30d, std_balance_30d,
    avg_balance_90d, std_balance_90d,
    total_inflow_30d, total_outflow_30d,
    total_inflow_90d, total_outflow_90d

    // --- 5. Risk History (49) ---
    days_since_last_alert
    // tier in {info, review, alert, block}, window in {24h, 7d, 30d, 90d}
    count_<tier>_<window>, confirmed_count_<tier>_<window>,
    rejected_count_<tier>_<window>

    // --- 6. Sequence Buffer (1) ---
    recent_events_buffer : list[event]    // bounded to 50, LPUSH+LTRIM
}

```

4.2 Cold Store (PostgreSQL) Tables

```

clients_master {
  client_id (PK), archetype, account_opening_date,
  nationality, client_type, home_branch_id, kyc_data
}

accounts {
  account_id (PK), client_id (FK), account_type,
  current_balance, current_debt, opening_date, status
}

account_balances {
  account_id (FK), snapshot_date, balance, debt
}

transactions {
  event_id (PK), client_id (FK), account_id (FK), employee_id (FK),
  branch_id, timestamp, amount, currency, operation_type,
  payload (JSON), status
}

alerts {
  alert_id (PK), event_id (FK), client_id (FK), employee_id (FK),
  timestamp, severity_tier, anomaly_score, shap_explanation,
  supervisor_decision, supervisor_notes, decision_timestamp
}

profile_snapshots { // optional, for LSTM training and audit
  client_id (FK), snapshot_date, profile_json
}

archetype_baselines { // for cold-start derived features
  archetype, window, mean_amount_<op>, std_amount_<op>,
  branch_distribution_pooled, hour_distribution_pooled, ...
}

```

5 Schema 3 — Employee Profile

5.1 Hot Profile (Redis) — 5 Categories, ~188 fields

```

employee_profile_hot {

  // --- 1. Personal Info (6) ---
  employee_id      : string  (PK)
  assigned_branch_id : string
  role             : enum    [teller, supervisor, ...]
  hire_date        : date
  last_transfer_date : date
  maturity_status   : enum    [transitioning, established]

  // --- 2. Transaction Statistics (113) ---
  // Same structure as client, for PROCESSED transactions
  days_since_last_processed_transaction
  stats_<op>_<window> { count, sum_amount, mean_amount, std_amount,
                        rejected_count, reversed_count,
                        near_threshold_ratio }
}

```

```

// --- 3. Behavioral Patterns (10 distributions) ---
behavior_30d {
  branch_distribution    // part-time / coverage may produce multiples
  hour_distribution
  dow_distribution
  operation_distribution
  client_distribution    // replaces employee_distribution
}
behavior_180d { ... same 5 distributions ... }

// --- 4. Peer Comparison (10) --- NEW vs client profile ---
peer_comparison_30d {
  z_volume, z_error_rate, z_near_threshold_ratio,
  z_client_concentration, z_client_location_mismatch
}
peer_comparison_90d { ... same 5 z-scores ... }

// --- 5. Risk History (49) ---
// Same structure as client; alerts involving the employee.
days_since_last_alert
count_<tier>_<window>, confirmed_count_<tier>_<window>,
rejected_count_<tier>_<window>
}

```

Financial state intentionally excluded: an employee’s personal balance is investigative HR / internal-audit territory, not transaction anomaly detection. Out of scope.

5.2 Cold Store Additions

```

employees_master {
  employee_id (PK), assigned_branch_id, role,
  hire_date, last_transfer_date, status
}

peer_baselines {
  // Per (branch_id, role); recomputed nightly by the batch pipeline.
  branch_id, role, snapshot_date,
  mean_volume_<window>, std_volume_<window>,
  mean_error_rate_<window>, std_error_rate_<window>,
  mean_near_threshold_ratio_<window>, std_near_threshold_ratio_<window>,
  mean_client_concentration_<window>, std_client_concentration_<window>,
  mean_client_location_mismatch_<window>,
  std_client_location_mismatch_<window>
  // for window in {30d, 90d}
}

```

6 Schema 4 — Enriched Event

Pattern: the Enrichment Service *enriches* (computes derived features) rather than merely *joins* (concatenates raw data). This keeps a clean separation of concerns with the Scoring Service. The result is ~64 features (plus an LSTM sequence buffer), down from ~382 raw profile fields.

```

enriched_event {

  // --- 1. Metadata & Traceability ---
  enriched_event_id      : string  (UUID, PK)

```

```

raw_event_id          : string (FK)
enrichment_timestamp  : datetime
raw_event_timestamp    : datetime
client_profile_snapshot_id : string
employee_profile_snapshot_id : string

// --- 2. Raw Event Passthrough ---
raw_event { ... full raw_event object ... }

// --- 3. Cold-Start & Maturity Flags ---
is_client_mature      : bool
is_employee_established : bool
client_archetype      : enum

// --- 4. Regulatory Rule Signals (profile-independent) ---
is_above_threshold    : bool // >= 10,000 DT
is_near_threshold_band : bool // 8,000 <= amount < 10,000
is_outside_business_hours : bool
is_outside_business_days : bool
has_duplicate_recent   : bool

// --- 5. Client-Side Derived Features ---
z_amount_30d, z_amount_90d
branch_frequency_30d, branch_frequency_180d
hour_frequency_30d, hour_frequency_180d
dow_frequency_30d, dow_frequency_180d
employee_frequency_30d, employee_frequency_180d
is_first_time_beneficiary, days_since_first_seen_beneficiary
is_first_time_emitter, days_since_first_seen_emitter
cumulative_amount_24h, cumulative_amount_7d
would_cross_threshold_24h, near_threshold_ratio_24h
current_balance, amount_vs_balance_ratio
z_balance_30d, inflow_vs_outflow_ratio_30d

// --- 6. Archetype Baseline Features (cold-start fallback) ---
z_amount_vs_archetype
branch_frequency_vs_archetype
hour_frequency_vs_archetype
dow_frequency_vs_archetype

// --- 7. Behavioral Drift Features ---
// JS-divergence between behavior_30d and behavior_180d
branch_drift_30d_vs_180d
hour_drift_30d_vs_180d
dow_drift_30d_vs_180d
operation_drift_30d_vs_180d
employee_drift_30d_vs_180d

// --- 8. Employee-Side Derived Features ---
z_amount_employee_30d, z_amount_employee_90d
branch_frequency_employee_30d, branch_frequency_employee_180d
hour_frequency_employee_30d, hour_frequency_employee_180d
client_frequency_employee_30d, client_frequency_employee_180d
client_location_mismatch : bool

// --- 9. Peer Comparison (passthrough from employee profile) ---
peer_z_volume_30d, peer_z_volume_90d
peer_z_error_rate_30d, peer_z_error_rate_90d

```

```

peer_z_near_threshold_ratio_30d, peer_z_near_threshold_ratio_90d
peer_z_client_concentration_30d, peer_z_client_concentration_90d
peer_z_client_location_mismatch_30d, peer_z_client_location_mismatch_90d

// --- 10. Risk History Signals ---
client_recent_alert_count_7d
client_confirmed_alert_count_30d
client_days_since_last_alert
employee_recent_alert_count_7d
employee_confirmed_alert_count_30d
employee_days_since_last_alert

// --- 11. LSTM Sequence Input ---
recent_events_sequence : list[event]    // up to 50, from client profile
}

```

6.1 Enriched Event — Feature Count Summary

Section	Features
3. Cold-start flags	3
4. Regulatory signals	5
5. Client-side derived	22
6. Archetype baseline	4
7. Behavioral drift	5
8. Employee-side derived	9
9. Peer comparison passthrough	10
10. Risk history signals	6
Total feature vector (Autoencoder input)	~64
LSTM sequence buffer	separate

7 Consolidated Open Questions

1. GAB (ATM) scope — pending supervisor confirmation.
2. Cheque-returned (NSF/bounce) event handling — lagging event design.
3. Serialization format — JSON for prototype; Avro/Protobuf for production with schema registry.
4. Balance source — daily snapshots vs balance-update events from core banking.
5. Cold-start thresholds (clients) and transitioning thresholds (employees) — calibrated during evaluation.
6. Profile snapshot cadence — nightly, weekly, or on-demand for LSTM training?
7. Distribution truncation — top-N or frequency threshold for unbounded distributions (branch, employee, client, beneficiary, emitter).
8. Peer set definition for small branches — fallback to region or role-only.
9. Peer baseline refresh cadence.
10. Peer set membership rules (recently transferred, part-time).

8 Next Step

With all four schemas defined, the next phase is implementation. The recommended order (per Checkpoint 3.1, batch-before-streaming):

1. Synthetic data generator (Faker/SDV) producing realistic `transactions` records respecting the 5 archetypes.
2. Batch pipeline — feature engineering and profile computation.
3. Model training — Autoencoder and LSTM.
4. Streaming services — Ingestion, Enrichment, Scoring, Decision.
5. Compliance Dashboard with supervisor feedback loop.